Integer and Floating-Point Representation

Algorithms, Programming, and Data Representation · Module 92313 · Chapters 3+4

From Fundamentals to AI Apocalypse · personal study guide

Contents

Start from memory 2

How to use this guide 2

How the chapters fit together 3

3.1 Number systems and positional notation 4

3.2 Integer representations 6

3.3 Character data 8

3.4 Unsigned binary arithmetic 9

3.5 Two's-complement arithmetic and overflow 10

4.1 The floating-point model 11

4.2 Binary fractions 12

4.3 Floating-point arithmetic and error 14

4.4 IEEE 754 binary formats 16

4.5 Zero, subnormal numbers, infinity, and NaN 18

Formula map 21

Derive it cold 22

Explain it 23

Source check 23

Mixed practice 24

Exam-style problems 26

Self-check 27

Answers 29

These chapters build from bit patterns and fixed-width integer rules to IEEE floating point. They finish by showing how finite precision affects calculations and how stable methods reduce error.

Start from memory

Answer before reading. Mark each response sure, unsure, or guessed.

1. Why must a signed bit pattern be interpreted with an encoding and width?
2. What is the n-bit two's-complement range?
3. Which floating-point field controls precision, and which controls range?
4. Why can subtracting close floating-point values produce a poor relative result?
5. What is the main accuracy advantage of fused multiply-add?

Answers: see "Start from memory" under Answers at the end.

How to use this guide

Try each question cold before reading the worked explanation. Rebuild the rule, then solve interleaved problems so you practise choosing a method.

1. **Day 0:** Recall §3.1–§3.2 and work practice 1–10.
2. **Day 1:** Rebuild character and integer arithmetic rules in §3.3–§3.5; solve practice 11–20.
3. **Day 3:** Recall §4.1–§4.3; solve practice 21–30.
4. **About day 7:** Rebuild IEEE decoding and numerical stability in §4.4–§4.5; solve practice 31–40.
5. **About day 14:** Complete both exam-style problems without notes.
6. **Before the exam:** Tick only skills you can demonstrate cold and revisit missed-answer references.

When stuck: write down the representation, width, and assumptions first. Then show each conversion or arithmetic step and check the result against its range or error bound.

How the chapters fit together

Integer representation explains exact bit-level meaning. Floating point extends this to a wide range of approximate real values. Numerical analysis then asks how representation and operation order affect the final answer.

Represent · assign meaning to bits

- **Positional bases** §3.1

  $\Sigma\, d_i b^i$

  Rests on: each digit's place has a power-of-base weight.

- **Integer encodings** §3.2

  bit pattern + width + rule

  Rests on: positional values and a fixed number of bits.

Calculate · preserve exact meaning

- **Fixed-width arithmetic** §3.5

  exact result → range check

  Rests on: the encoding's representable range.

- **Floating point** §4.1

  $(-1)^s \times \text{significand} \times 2^e$

  Rests on: sign, biased exponent, and finite precision.

Evaluate · bound numerical error

- **Rounding and spacing** §4.3

  $\text{fl}(x)=x(1+\delta)$

  Rests on: a finite set of representable values.

- **Numerical stability** §4.5

  reformulate before precision is lost

  Rests on: tracking where rounding error enters and how operations propagate it.

Question	Method
What does a bit pattern represent?	State width, encoding, and format.
Is a fixed-width result valid?	Compute exactly, then check its representable range.
Why did a float change?	Inspect spacing, rounding, range, and operation order.
Can cancellation be avoided?	Reformulate the calculation or use one-rounding operations.

3.1 Number systems and positional notation

A numeral is a compact way to write a sum of place values. Binary, octal, decimal, and hexadecimal use the same positional rule with different bases.

Expand and convert

In base b, each digit d_i contributes $d_i b^i$, where the rightmost digit has i=0. Binary digits are 0 or 1. Decimal-to-binary repeated division records remainders; read them from last to first.

Positional value

$$(d_k \ldots d_1 d_0)_b = \sum d_i b^i$$

$$(1100010)_2 = 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^1 = 98$$

The worked binary number is 98 in decimal. Subscripts identify the base.

Worked conversion · decimal to binary

Convert 45 to binary by repeated division.

1. $45 \div 2$ gives quotient 22, remainder 1.
2. $22 \div 2$ gives quotient 11, remainder 0; then 11 gives 5 remainder 1.
3. Continue: 5 gives 2 remainder 1; 2 gives 1 remainder 0; 1 gives 0 remainder 1.
4. Read remainders upward: 101101_2. Check: 32+8+4+1=45. §3.1

Octal and hexadecimal

Because $8=2^3$, each octal digit corresponds to three binary digits. Because $16=2^4$, each hexadecimal digit corresponds to four bits. Group binary digits from the right and pad the leftmost group with zeroes if needed. In hexadecimal, A–F mean 10–15.

Compact binary groups

$$10101_2 = 010\,101 = 25_8$$
$$10010100_2 = 1001\,0100 = 94_{16}$$
$$1\,\text{byte} = 8\,\text{bits} = 2\,\text{hexadecimal digits}$$

The lecture uses 16-bit words and 32-bit long words as examples; the sizes named by these terms can depend on a computer architecture.

Trap: do not read a binary string as an ordinary decimal integer. Attach its base, expand it, and only then compare values.

Recall check · answer without looking

1. What power of the base multiplies the rightmost digit?
2. Why can binary be grouped in threes to convert to octal?
3. What is A in hexadecimal worth in decimal?

Answers: see "3.1 Number systems and positional notation" under Answers at the end.

3.2 Integer representations

A computer stores a fixed number of bits, not a plus or minus sign unless the encoding defines one. The same pattern can mean different values under different encodings.

Unsigned and signed ranges

An unsigned n-bit pattern represents 0 through 2^n-1. Signed encodings trade positive range for negative values. The source compares sign-and-magnitude, one's complement, two's complement, excess (biased) notation, and binary-coded decimal (BCD).

Ranges for n bits

$$\text{Unsigned}: 0 \dots 2^n - 1$$
$$\text{Sign and magnitude}: -(2^{n-1}-1) \dots +(2^{n-1}-1)$$
$$\text{One's complement}: -(2^{n-1}-1) \dots +(2^{n-1}-1)$$
$$\text{Two's complement}: -2^{n-1} \dots 2^{n-1}-1$$

Sign-and-magnitude and one's complement have two zero encodings. Two's complement has one zero and one extra negative value.

Worked comparison · decode 1111 in four encodings

1. Unsigned: 15.
2. Sign-and-magnitude: sign bit 1 and magnitude 111, so −7.
3. One's complement: invert to 0000, so −0, which equals zero but has a distinct pattern.
4. Two's complement: the unsigned value 15 represents 15−16=−1. §3.2

Two's complement and bias

To form −x in n-bit two's complement, invert all n bits of x and add 1; discard a carry beyond the width. The minimum value -2^{n-1} is the exception: its positive counterpart is outside the signed range, so fixed-width negation leaves the same pattern and overflows. Equivalently, interpret a pattern with top bit 1 by subtracting 2^n from its unsigned value. In excess-B notation, actual value = stored unsigned value − B. The lecture's three-bit excess-4 example therefore maps 000 to −4 and 111 to 3.

Decode rules

$$\text{Two's}-\text{complement value} = \text{unsigned(pattern)} - 2^n \text{ when the top bit is 1}$$
$$\text{Negate x}: \text{invert n bits, then add 1}$$
$$\text{Excess}-\text{B value} = \text{stored value} - \text{B}$$

Keep the width fixed throughout. The same bit pattern has no signed meaning until an encoding is specified.

BCD stores decimal digits separately

In four-bit BCD, each decimal digit has its own four-bit code. For example, decimal 59 is 0101 1001 in BCD, while the ordinary binary value 59 is 00111011. BCD uses more bits but keeps each decimal digit directly readable.

Trap: do not convert all BCD bits as one binary number. Decode each four-bit group as a decimal digit.

Recall check · answer without looking

1. What is the four-bit two's-complement range?
2. Why is the two's-complement range asymmetric?
3. What is the BCD encoding of decimal 42?

Answers: see "3.2 Integer representations" under Answers at the end.

3.3 Character data

Numbers are not the only data stored as bits. A character encoding assigns numeric codes to letters, digits, punctuation, and control characters.

ASCII and Unicode

ASCII uses seven bits, giving 128 code values. The lecture describes eight-bit extensions that use the additional 128 patterns, but those upper values depend on the particular extension. Unicode assigns code points to characters across many writing systems; its first 128 code points match ASCII. A code point and its byte encoding are related but are not the same concept.

Character-code facts

$$7 - \text{bit ASCII} : 2^7 = 128 \text{ code values}$$
$$\text{ASCII code points} : 0 \ldots 127$$

The first 128 Unicode code points correspond to ASCII. Byte encodings represent code points as bytes by their own rules.

Worked encoding · read a short ASCII string

The lecture's example encodes "Fred" in hexadecimal as 46 72 65 64.

1. Split the bytes: 46, 72, 65, 64.
2. Read each as a hexadecimal ASCII code: 46 is F, 72 is r, 65 is e, and 64 is d.
3. The four codes decode to "Fred". §3.3

Trap: an encoding is a mapping, not a property of the bit pattern by itself. Identify the character set and any byte encoding before decoding text.

Recall check · answer without looking

1. How many code values are available in seven-bit ASCII?
2. What is the relation between ASCII and the first 128 Unicode code points?

Answers: see "3.3 Character data" under Answers at the end.

3.4 Unsigned binary arithmetic

Binary arithmetic follows the same place-value logic as decimal arithmetic. Carries and borrows move between columns, while the fixed bit width limits which values can be stored.

Add and subtract

Add each column and carry 1 when a column totals 2. For subtraction, borrow from a higher column when the current minuend bit is too small; a borrowed binary 1 is worth 2 in the current column. Subtraction can also be checked by adding the difference to the subtrahend.

Worked addition · include the carry

Add 111011_2 and 101010_2.

1. From right to left: 1+0=1.
2. 1+1=10, so write 0 and carry 1. Continue each column with its carry.
3. The result is 1100101_2. Check: 59+42=101, and 1100101_2=101. §3.4

Multiply and divide

Binary multiplication uses partial products, like decimal long multiplication, but each multiplier bit is only 0 or 1. Binary division uses quotient and remainder. For divisor $d \neq 0$, dividend = quotient×divisor + remainder, where $0 \leq \text{remainder} < \text{divisor}$.

Division check

$$\text{dividend} = \text{quotient} \times \text{divisor} + \text{remainder}$$
$$0 \leq \text{remainder} < \text{divisor}$$

Substitute the quotient, divisor, and remainder to check a division result.

Memory hook: in binary addition, 1+1 writes 0 and carries 1. In subtraction, a borrow makes the current column worth 2 before subtracting.

Recall check · answer without looking

1. What bit do you write for 1+1, and what do you carry?
2. What inequality must the remainder satisfy?

Answers: see "3.4 Unsigned binary arithmetic" under Answers at the end.

3.5 Two's-complement arithmetic and overflow

Two's complement lets the same adder perform signed addition and subtraction. Correct bit operations still need a range check: a fixed-width result can wrap to a pattern with a different signed value.

Addition, subtraction, and overflow

Add the bit patterns and discard any carry beyond the most significant bit. For subtraction, negate the subtrahend and add it to the minuend. Signed overflow occurs when the exact mathematical result lies outside the representation's range. For addition, it occurs when two operands with the same sign produce a result with the opposite sign; operands of unlike signs cannot overflow.

n-bit two's-complement checks

$$\text{Range}: -2^{n-1} \ldots 2^{n-1} - 1$$

$$A - B = A + (\text{two's} - \text{complement negation of } B)$$

$$\text{Same} - \text{sign addition with opposite} - \text{sign result} \Rightarrow \text{overflow}$$

Discarding carry-out is part of fixed-width addition. Carry-out alone does not signal signed overflow.

Worked example · detect overflow

Use four-bit two's complement to calculate $-7+(-6)$.

1. Encode -7 as 1001 and -6 as 1010.
2. Add: 1001+1010=1 0011. Discard the carry-out, leaving 0011.
3. The stored result 0011 is $+3$, but the exact sum is -13, outside the four-bit range $-8\ldots+7$. Both inputs were negative and the result is positive, so overflow occurred. §3.5

Multiplication and division

The lecture notes that ordinary unsigned multiplication cannot simply be applied to signed two's-complement patterns and read at the original width. A full product may require up to twice as many bits as the operands; retain a sufficient width and interpret the result consistently. For a simple signed multiplication, multiply magnitudes and make the result negative exactly when the operand signs differ, then check the available width. The supplied slides introduce this idea but do not give a complete signed division algorithm.

Sign check: do not treat discarded carry as overflow. Decode the fixed-width result and compare it with the exact mathematical result and allowed range.

Recall check · answer without looking

1. How is A−B implemented in two's-complement arithmetic?
2. Can two's-complement addition overflow when the operands have unlike signs?
3. Why can a signed product need more bits than either input?

Answers: see "3.5 Two's-complement arithmetic and overflow" under Answers at the end.

4.1 The floating-point model

A fixed number of bits cannot represent every real number.

Floating point uses a sign, a significand, and an exponent to cover very large and very small magnitudes.

Range and precision

For base β, a normalised number has the form sign × significand × β^{exponent}.

The exponent controls scale and therefore the approximate range.

The number of significand bits controls precision, or how finely nearby values can be distinguished.

Floating-point values are finite in number, and their spacing changes with magnitude.

Scientific notation model

$$x = s \times M \times \beta^{E}$$

s is the sign, M is the significand, β is the base or radix, and E is the exponent. In binary floating point, β=2.

Worked example · normalise a decimal value

Write 343000×10^{-4} in normalised decimal form with $1 \leq M < 10$.

1. Move the decimal point in 343000 five places left to make 3.43.
2. Increase the exponent by five to preserve the value: 3.43×10^{1}.
3. Check: $343000 \times 10^{-4} = 34.3$, and $3.43 \times 10^{1} = 34.3$. §4.1

Trap: moving the radix point changes the exponent in the opposite direction if the represented value is to stay the same.

Recall check · answer without looking

1. Which field chiefly determines range?
2. Which field chiefly determines precision?
3. Why are floating-point values not evenly spaced over the number line?

Answers: see "4.1 The floating-point model" under Answers at the end.

4.2 Binary fractions

Binary fractional place values explain both conversion and approximation.

The first bit after the binary point has weight 2^{-1}.

Each following bit has half the previous weight.

Convert binary fractions to decimal

Binary fractional place values

$$0.b_1b_2...b_k = b_1 2^{-1} + b_2 2^{-2} + ... + b_k 2^{-k}$$

Each b_i is 0 or 1. A leading integer part is evaluated with non-negative powers of 2.

Worked conversion · binary fraction to decimal

Convert 0.01101_2 to decimal.

1. Use the five place values: 1/2, 1/4, 1/8, 1/16, and 1/32.
2. Keep the places where the bit is 1: 1/4 + 1/8 + 1/32.
3. Add: 0.25+0.125+0.03125=0.40625. §4.2

Convert decimal fractions to binary

Repeatedly multiply the fractional part by 2.

At each step, the integer part is the next binary digit, and the new fractional part is carried into the next step.

Worked conversion · terminating binary fraction

Convert 0.6875 to binary.

1. $0.6875 \times 2 = 1.375$, so the first bit is 1; carry 0.375.
2. $0.375 \times 2 = 0.75$, so the next bit is 0; carry 0.75.
3. $0.75 \times 2 = 1.5$, so the next bit is 1; carry 0.5.
4. $0.5 \times 2 = 1.0$, so the next bit is 1; the fraction is now zero.
5. Therefore $0.6875 = 0.1011_2$. Check: $1/2 + 1/8 + 1/16 = 0.6875$. §4.2

Some fractions, such as 0.1 in decimal, repeat in binary.

A finite significand stores only an approximation of a repeating expansion.

Trap: a decimal fraction that looks short is not necessarily exact in binary.

Recall check · answer without looking

1. What is the weight of the fourth binary digit after the point?
2. What is 0.101_2 in decimal?
3. Why can a computer store 0.1 only approximately in binary floating point?

Answers: see "4.2 Binary fractions" under Answers at the end.

4.3 Floating-point arithmetic and error

The exponent and significand must be managed during each operation.

After the exact operation, the result may need normalisation and rounding to fit the available precision.

Multiplication

Multiply normalised values

$$(M_1 \times \beta^{E_1})(M_2 \times \beta^{E_2}) = (M_1 M_2) \times \beta^{E_1+E_2}$$

Multiply the significands and add the exponents. Then normalise and round if needed.

Worked multiplication · normalise the result

Calculate $(2.6\times10^{3})(5.4\times10^{-2})$.

1. Multiply significands: $2.6\times5.4=14.04$.
2. Add exponents: $3+(-2)=1$, giving 14.04×10^{1}.
3. Normalise: $1.404\times10^{2}=140.4$. §4.3

Addition

Before adding, express both numbers with the same exponent.

Shift the significand of the value with the smaller exponent to the right by the exponent difference.

This alignment can discard low-order digits before the addition is even performed.

Worked addition · align exponents

Use two significant decimal digits to add 4.5×10^{3} and 6.7×10^{2}.

1. Rewrite the second value at exponent 3: $6.7\times10^{2}=0.67\times10^{3}$.
2. Add significands: $(4.5+0.67)\times10^{3}=5.17\times10^{3}$.
3. Round to two significant digits: 5.2×10^{3}. The exact sum is 5170, while this two-digit result is 5200. §4.3

Rounding, overflow, and comparison

Truncation drops extra digits and tends to bias results in one direction.

Rounding chooses a nearby representable value and usually gives a better approximation.

Exponent overflow occurs when the result is too large for the format.

Exponent underflow occurs when the result is too small for the normalised range.

The slides describe infinity or an exception for overflow, and zero or an exception for underflow in their simplified model.

IEEE formats also provide subnormal numbers near zero.

For approximate comparisons, choose a tolerance ε that fits the scale and precision of the problem.

Absolute tolerance example

$$\text{Treat a as close to b when } |a - b| < \varepsilon$$

$$\text{For } b = 1 \text{ and } \varepsilon = 0.000005 : \; 0.999995 < a < 1.000005$$

A fixed absolute tolerance may be unsuitable when magnitudes vary greatly. The slides also introduce relative closeness.

Sign check: estimate the order of magnitude before accepting a rounded result.

Recall check · answer without looking

1. What do you do to the exponents when multiplying?
2. Which operand's significand is shifted to align an addition?
3. How do overflow and underflow differ?
4. Why can direct equality comparison be unreliable for computed floating-point values?

Answers: see "4.3 Floating-point arithmetic and error" under Answers at the end.

4.4 IEEE 754 binary formats

A shared format defines how floating-point fields are stored and how arithmetic behaves.

The slides introduce IEEE 754 as a widely adopted standard for predictable binary floating-point arithmetic.

Single and double precision

Format	Total	Sign	Exponent	Fraction field	Significand precision	Bias
Single	32 bits	1 bit	8 bits	23 bits	24 bits for normal values	127
Double	64 bits	1 bit	11 bits	52 bits	53 bits for normal values	1023

For a normalised binary value, the leading significand bit is 1 and is omitted from the stored fraction field.

The exponent field stores a biased exponent rather than a two's-complement exponent.

Normalised value rule

$$\text{value} = (-1)^{S} \times 1.F \times 2^{E-\text{bias}}$$

$$\text{Single bias} = 127;\ \text{double bias} = 1023$$

S is the sign bit, E is the unsigned exponent field, and F is the stored fraction interpreted after the binary point. This rule is for normalised values.

Convert decimal to single precision

Worked conversion · 42.6875

1. Convert to binary: $42.6875 = 101010.1011_2$.
2. Normalise: 1.010101011×2^5, so the sign bit is 0.
3. Store the fraction after the hidden leading 1: 01010101100000000000000.
4. Bias the exponent: $5 + 127 = 132 = 10000100_2$.
5. The fields are 0 | 10000100 | 01010101100000000000000, or hexadecimal 422A C000. §4.4

Decode single precision

Worked decoding · BEC0 0000

1. The sign bit is 1, so the value is negative.
2. The exponent field is 01111101_2=125; subtract the bias: 125−127=−2.
3. Restore the hidden bit: $1.1000\ldots_2$=1.5.
4. Scale: 1.5×2^{-2}=0.375. Apply the sign: −0.375. §4.4

Trap: restore the hidden leading 1 when decoding a normalised value, but do not apply that rule to a subnormal value.

Recall check · answer without looking

1. How many bits are stored in the single-precision fraction field?
2. What is the single-precision exponent bias?
3. Why can a normalised value have 24 bits of significand precision while storing only 23 fraction bits?

Answers: see "4.4 IEEE 754 binary formats" under Answers at the end.

4.5 Zero, subnormal numbers, infinity, and NaN

Reserved exponent patterns extend useful behaviour beyond normalised values.

In single precision, exponent field zero and exponent field all ones have special interpretations.

Exponent field	Fraction field	Meaning
All zeroes	All zeroes	Positive or negative zero
All zeroes	Non-zero	Subnormal number
Neither all zeroes nor all ones	Any pattern	Normalised number
All ones	All zeroes	Positive or negative infinity
All ones	Non-zero	NaN, or not-a-number

For a subnormal single-precision number, the leading bit is 0 and the effective exponent is -126.

Subnormal values provide a gradual step from the smallest normalised values toward zero.

Infinity represents overflow results and some divisions by zero.

NaN represents an operation without a real-number result, such as $0/0$ or infinity−infinity.

Operations on infinity and NaN follow format rules, not ordinary real-number algebra in every case.

Special arithmetic examples

Operation	Result
$N \div \pm\infty$	0
$\pm$ non-zero $\div 0$	$\pm\infty$
$+\infty + +\infty$	$+\infty$
$\pm\infty \times \pm\infty$	$\pm\infty$
$\pm 0 \div \pm 0$	NaN
$+\infty - +\infty$	NaN
$\pm\infty \div \pm\infty$	NaN
$\pm\infty \times 0$	NaN

The sign of the infinity in signed cases depends on the operand signs. Slides note that an invalid operation may return a quiet NaN or raise a signalling-NaN exception.

> **Worked classification · single precision fields**
> Classify sign 0, exponent 11111111, fraction 000…000.
>
> 1. The exponent field is all ones.
> 2. The fraction field is all zeroes.
> 3. This encodes positive infinity. §4.5

Trap: an all-ones exponent does not always mean infinity; inspect the fraction field to distinguish infinity from NaN.

Numerical stability and cancellation

Cancellation occurs when subtracting close stored values. The subtraction may itself be exact, yet earlier rounding in the operands can dominate the small difference. A stable method avoids needlessly creating that sensitive subtraction.

Machine epsilon and relative error

Machine epsilon is the gap from 1 to the next larger representable value. For radix β and p significant digits, $\varepsilon_{\text{mach}}=\beta^{1-p}$. Here p counts the full significand digits, including the hidden leading bit in a normalised IEEE value.

Binary machine epsilon

$$\text{Single precision}: \ p = 24,\ \varepsilon_{\text{mach}} = 2^{-23} \approx 1.19 \times 10^{-7}$$
$$\text{Double precision}: \ p = 53,\ \varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$$

Under round-to-nearest, the unit roundoff u is $\varepsilon_{\text{mach}}/2$ for normal results.

Relative rounding model

$$\text{fl}(x) = x(1+\delta),\ \text{with}\ |\delta| \le u$$

For finite non-zero normal results, round-to-nearest, and no overflow. Relative error is measured against $|x|$, so the same absolute error matters more when the true answer is small.

Worked example · cancellation

Suppose five significant decimal digits store x=0.123456789 as 0.12346 and y=0.123451234 as 0.12345.

1. The exact difference is 0.000005555.
2. The stored difference is 0.00001.
3. The stored difference is about 1.80 times the exact difference, so its relative error is about 80.0%. The subtraction exposed input rounding error. §4.5

Sterbenz lemma

If x and y are positive finite floating-point values in the same radix and precision, and $y/2 \le x \le 2y$, then $x-y$ is exactly representable when gradual underflow is supported. For example, 1.5 and 1.0 meet the condition, and their difference 0.5 is exact. This does not undo rounding that happened while x or y was computed; cancellation can expose that earlier error.

Stable quadratic roots

For $ax^2+bx+c=0$, require $a\neq 0$ and $D=b^2-4ac\geq 0$. The usual formula can subtract nearly equal quantities for one root. For $D>0$, choose the branch that adds magnitudes: if $b\geq 0$, set $q=-\frac{1}{2}(b+\sqrt{D})$; if $b<0$, set $q=-\frac{1}{2}(b-\sqrt{D})$. Then the roots are q/a and c/q. This branch also handles $b=0$. For $D=0$, use the repeated root $-b/(2a)$, since q may be zero. These steps avoid cancellation while respecting the degenerate cases.

Trap: mathematical equivalence does not guarantee equal numerical accuracy. Look for subtraction of nearly equal values and avoid rounding intermediate results unnecessarily.

Source qualification: the lecture's performance statements about subnormals and its description of C signed-char conversion depend on implementation or execution mode. Follow the language and platform rules for a specific system.

Fused multiply-add

A fused multiply-add evaluates $a\times b+c$ with one final rounding. Separate multiplication and addition can round the product first, then round again after addition. FMA can therefore preserve more information; its speed benefit depends on hardware.

Fused versus separate rounding

$$\mathrm{FMA}(a,b,c)=\mathrm{round}(a\times b+c)$$
$$\text{Separate}:\ \mathrm{round}(\mathrm{round}(a\times b)+c)$$

The fused operation avoids the intermediate rounding step.

Recall check · answer without looking

1. How do you distinguish infinity from NaN?
2. What is the main purpose of subnormal numbers?
3. What does zero exponent and zero fraction encode?

Answers: see "4.5 Zero, subnormal numbers, infinity, and NaN" under Answers at the end.

Formula map

Idea	Relation	Use
Positional value	$\Sigma d_i b^i$	Decode a numeral.
Unsigned n-bit range	$0 \dots 2^n-1$	Check fit.
Two's-complement range	$-2^{n-1} \dots 2^{n-1}-1$	Check signed fit.
Biased exponent	actual exponent = stored exponent − bias	Decode a normal IEEE value.
Floating-point model	$(-1)^S \times \text{significand} \times 2^E$	Interpret sign, scale, and precision.
Machine epsilon	β^{1-p}	Spacing from 1 to the next value; p is the total number of significand digits.
Unit roundoff	$u=\varepsilon_{mach}/2$	Bound relative rounding error for normal results under round-to-nearest.
Relative error model	$fl(x)=x(1+\delta)$, $\lvert\delta\rvert\leq u$	Model rounding under stated assumptions.
Sterbenz condition	$y/2 \leq x \leq 2y$	Subtraction of such floating-point x and y is exact under the lemma's assumptions.

Check every answer

State width and encoding for integer problems

Track signs, exponent alignment, and rounding

Check limiting cases and exact ranges

Integer problems have no physical units. In floating-point problems, report the numerical precision and assumptions used.

Derive it cold

Cover the explanations and rebuild each result on blank paper.

1. **Base conversion:** expand a binary numeral into powers of 2. §3.1
2. **Binary to octal/hex:** derive why groups of 3/4 bits match one digit. §3.1
3. **Two's-complement negation:** show why invert-and-add-one gives $2^n - x$. §3.2
4. **Signed range:** determine the lowest and highest n-bit two's-complement values. §3.2
5. **Unsigned division:** verify a quotient by reconstructing the dividend. §3.4
6. **Addition overflow:** explain why same-sign inputs can produce an invalid opposite-sign result. §3.5
7. **Subtraction:** replace $A - B$ with $A + (-B)$ and show the bit width at every step. §3.5
8. **Binary fractions:** derive each bit's place value as a negative power of 2. §4.2
9. **Decimal fraction conversion:** use repeated multiplication by 2 and track the fractional remainder. §4.2
10. **Multiplication:** use exponent laws, then normalise. §4.3
11. **Addition:** align exponents, combine significands, and round. §4.3
12. **Single-precision decoding:** recover the sign, unbiased exponent, and hidden bit. §4.4
13. **Special values:** derive the classification from exponent and fraction fields. §4.5
14. **Cancellation and stable roots:** explain how input error is amplified and derive the product relation for the second quadratic root. §4.5
15. **One-rounding FMA:** compare fused evaluation with separately rounded multiply and add. §4.5
16. **Machine epsilon:** derive the gap near 1 from radix and significand precision. §4.5
17. **Sterbenz lemma:** check the ratio condition and distinguish exact subtraction from prior input rounding. §4.5

Explain it

Answer each why or when aloud before opening the model response.

1. Why does the rightmost binary digit have weight 1?
2. Why are octal and hexadecimal convenient ways to write binary?
3. Why do sign-and-magnitude and one's complement have two zero patterns?
4. Why does two's complement have one extra negative value?
5. When is carry-out discarded, and why is that not enough to detect signed overflow?
6. Why can BCD be easier for decimal display but less space-efficient?
7. Why does increasing the exponent change range more than precision?
8. Why can two nearby real values have different spacing when stored at different magnitudes?
9. Why does 0.1 decimal not have a finite binary fraction?
10. Why must exponents be aligned before adding two floating-point values?
11. Why can rounding during addition lose a small addend?
12. Why is the leading 1 omitted for normalised binary significands?
13. When would a subnormal number be used?
14. Why should an approximate comparison use a tolerance chosen for the problem?
15. Why is machine epsilon a relative-precision scale rather than one fixed absolute error for every number?
16. When can Sterbenz's lemma say the subtraction itself is exact, and what error can still remain?

Answers: see "Explain it" under Answers at the end.

Source check

These checks are inherited from the chapter guides and are tied to the printed page numbers in their source notes.

- **Chapter 3 slide 24:** the four-bit Excess-8 row for 0111 is printed as 1; stored value 7 minus bias 8 gives −1. §3.2
- **Extended lecture slide 6:** the C example says assigning 127+1 to signed char wraps to −128. That is not guaranteed by C: integer promotion computes 128 as int, then converting an out-of-range value to signed char has an implementation-defined result or signal. Do not generalise this example as signed integer overflow being defined wraparound. See clauses 6.3.1.1 and 6.3.1.3 of the WG14 C standard draft N1570.
- **Extended lecture slide 22:** the second quadratic root uses Vieta's relation $x_1x_2=c/a$, so $x_2=c/(a\ x_1)$; keep the parentheses explicit.

Mixed practice

Try all 47 problems before opening the worked answers. Interleaving makes you choose the representation or method each time.

1. Convert 37_{10} to binary and check by place value. §3.1
2. Convert 0.1011_2 to decimal. §4.2
3. Decode 1110 as a four-bit two's-complement number. §3.2
4. How many bits of significand precision does normalised IEEE single precision provide? §4.4
5. How many bits represent one hexadecimal digit? §3.1
6. Normalise 0.000101_2 as $M \times 2^E$ with $1 \le M < 2$. §4.1
7. Encode decimal 73 in four-bit BCD. §3.2
8. Convert 0.375 decimal to binary. §4.2
9. How many values are available in seven-bit ASCII? §3.3
10. In a floating-point format, which field chiefly determines range? §4.1
11. Add 1011_2 and 0110_2 as unsigned values. §3.4
12. Multiply $(3 \times 10^2)(2 \times 10^{-4})$ and give a normalised result. §4.3
13. What is the four-bit two's-complement range? §3.2
14. Align 7.2×10^5 to exponent 7. §4.3
15. Convert 101101_2 to octal. §3.1
16. How many stored fraction bits does double precision have? §4.4
17. In excess-4, what value does the three-bit pattern 110 represent? §3.2
18. Classify single-precision fields S=1, E=all ones, F=0. §4.5
19. For $29 \div 5$, give quotient and remainder and verify the division identity. §3.4
20. Convert 0.01101_2 to decimal. §4.2
21. Negate 0011 in four-bit two's complement. §3.2
22. Use two significant decimal digits to round $8.46 \times 10^3 + 2.1 \times 10^2$. §4.3
23. Convert 11101010_2 to hexadecimal. §3.1
24. What is the single-precision exponent bias? §4.4
25. Why does two's-complement addition discard a carry-out? §3.5
26. Convert decimal 0.625 to binary. §4.2
27. Write decimal 59 in BCD and ordinary binary. §3.2
28. For normalised single precision with exponent field E=130, what is the unbiased exponent? §4.4
29. Compute $7-(-6)$ using four-bit two's complement. State the exact result and whether it fits. §3.5
30. State one reason truncation can be less desirable than rounding. §4.3
31. What do the first 128 Unicode code points have in common with ASCII? §3.3
32. Classify single-precision fields E=0 and F non-zero. §4.5

33. Multiply 101_2 by 11_2 as unsigned values. §3.4
34. For 1.25×2^3, give the decimal value. §4.1
35. In four-bit two's complement, add 0101 and 0011. Does it overflow? §3.5
36. How many total bits are used in double precision? §4.4
37. What is the decoded value of pattern 0111 in four-bit excess-8? §3.2
38. Give an absolute-tolerance test for a value a to be close to b using ε. §4.3
39. What is one reason BCD can be useful despite its extra storage? §3.2
40. Convert 1.101_2 to decimal. §4.2
41. Why is 1.25×2^3 an exact binary value? §4.2
42. What do $\pm$non-zero $\div$ 0 and $\pm0 \div \pm0$ produce? §4.5
43. What happens to the exponent when multiplying two floating-point values? §4.3
44. When the exponent field is all ones and the fraction is zero, what is the value when S=0? §4.5
45. For binary single precision with 24 significand bits, give machine epsilon and unit roundoff. §4.5
46. Convert 53_{10} to binary and verify it by place value. §3.1
47. Check Sterbenz's condition for x=1.5 and y=1.0, then state their exact difference. §4.5

Answers: see "Mixed practice" under Answers at the end.

Exam-style problems

Allow about 20 minutes for each problem. Attempt each part before opening its worked answer.

1 · Compare encodings

1. Write the four-bit pattern in each encoding.
2. State the range and number of zero patterns for each representation.
3. Explain why two's complement is useful for arithmetic despite its asymmetric range.

Answers: see "Exam-style problems" under Answers at the end.

2 · Convert, calculate, and check

1. Write both input patterns and add them, showing any carry-out.
2. Decode the retained four-bit result and compare it with the exact sum.
3. State whether overflow occurred and explain why carry-out is not the test.

Answers: see "Exam-style problems" under Answers at the end.

1 · Encode and decode single precision

1. Convert 13.25 to binary.
2. Normalise it and identify sign, exponent, and stored fraction.
3. Give the exponent field and verify by decoding.

Answers: see "Exam-style problems" under Answers at the end.

2 · Align, add, and assess precision

1. Align both values to exponent 4.
2. Add the significands and round to two significant digits.
3. Compare the rounded result with the exact decimal sum and explain the source of the difference.

Answers: see "Exam-style problems" under Answers at the end.

Self-check

Tick a skill only in a later session, at least one day after studying, when you can demonstrate it without notes.

- ☐ Expand numbers by powers of their base. (1, 15, 23)
- ☐ Convert binary groups to octal and hexadecimal. (5, 15, 23)
- ☐ State signed and unsigned ranges from a bit width. (13)
- ☐ Decode patterns using distinct signed encodings. (3, 37)
- ☐ Negate a two's-complement value at fixed width. (21)
- ☐ Decode an excess-B value. (17, 37)
- ☐ Write and read decimal digits in BCD. (7, 27, 39)
- ☐ Explain ASCII and Unicode code-point relationships. (9, 31)
- ☐ Add binary values and carry correctly. (11, 33)
- ☐ Verify quotient and remainder. (19)
- ☐ Add signed values and detect overflow. (25, 35)
- ☐ Subtract by negating and adding. (29)
- ☐ Identify and correct the slide 24 excess-8 typo. (37)
- ☐ Rebuild the derivations from the formula list.
- ☐ Complete both exam-style problems with checks.
- ☐ Explain how significand and exponent affect precision and range. (6, 10)
- ☐ Normalise a floating-point value and preserve its magnitude. (6)
- ☐ Convert binary fractions to decimal. (2, 20, 40)
- ☐ Convert terminating decimal fractions to binary. (8, 26)
- ☐ Explain why some decimal fractions repeat in binary. (41)
- ☐ Multiply floating-point numbers and normalise. (12)
- ☐ Align exponents before adding. (14, 22, 44)
- ☐ Round results to a stated precision. (22, 30, 44)
- ☐ State an approximate comparison using tolerance. (38)
- ☐ State the single-precision field sizes and bias. (4, 24)
- ☐ Encode a decimal value in single precision. (exam 1)
- ☐ Decode a normalised single-precision value. (14, exam 1)
- ☐ Classify a subnormal pattern and explain its role. (32)
- ☐ Classify special-value fields and predict basic infinity/NaN arithmetic. (18, 42, 44)
- ☐ Rebuild the derivations from the list.
- ☐ Complete both exam-style problems with checks.
- ☐ Calculate binary machine epsilon and unit roundoff. (45)

- □ Apply Sterbenz's condition and explain remaining input error. (47)

Answers

Start from memory

1. The same bits can represent different values under different widths and encoding rules. §3.2
2. -2^{n-1} through $2^{n-1}-1$. §3.2
3. The significand field controls precision; the exponent field chiefly controls range. §4.1
4. Earlier rounding errors become large compared with a small difference. §4.5
5. It rounds the product-plus-sum once, avoiding an intermediate rounding. §4.5

3.1 Number systems and positional notation

1. The zeroth power, which is 1. §3.1
2. Each octal digit represents 0–7, exactly the range of three bits, and $8=2^3$. §3.1
3. 10. §3.1

3.2 Integer representations

1. -8 through $+7$. §3.2
2. There is one zero and one additional negative pattern, 1000, representing -8. §3.2
3. 0100 0010, one four-bit code for 4 and one for 2. §3.2

3.3 Character data

1. $2^7=128$. §3.3
2. The Unicode code points 0–127 correspond to the ASCII characters; their byte representation depends on the encoding used. §3.3

3.4 Unsigned binary arithmetic

1. Write 0 and carry 1 to the next column. §3.4
2. For a positive divisor, $0 \le \text{remainder} < \text{divisor}$. §3.4

3.5 Two's-complement arithmetic and overflow

1. Negate B in the chosen width, then add it to A and discard carry-out. §3.5
2. No. Overflow in addition requires two operands with the same sign and a result of the opposite sign. §3.5
3. The product magnitude can exceed the range of either input width; a full-width product may need up to twice as many bits. §3.5

4.1 The floating-point model

1. The exponent field. §4.1
2. The significand field. §4.1
3. Each exponent scales the significand by a different power of the base, so the gap between representable values grows with magnitude. §4.1

4.2 Binary fractions

1. $2^{-4}=1/16$. §4.2
2. $1/2+1/8=0.625$. §4.2
3. The binary expansion repeats, but the significand has a finite number of bits and must be rounded or truncated. §4.2

4.3 Floating-point arithmetic and error

1. Add them. §4.3
2. Shift the significand belonging to the smaller exponent to the right. §4.3
3. Overflow exceeds the maximum magnitude; underflow falls below the smallest normalised magnitude. §4.3
4. Finite precision can make a computed result differ slightly from its exact mathematical value, so a suitable tolerance may be needed. §4.3

4.4 IEEE 754 binary formats

1. Twenty-three. §4.4
2. 127. §4.4
3. The leading binary 1 is implicit, giving 24 significant bits including the 23 stored fraction bits. §4.4

4.5 Zero, subnormal numbers, infinity, and NaN

1. Both use an all-ones exponent. A zero fraction means infinity; a non-zero fraction means NaN. §4.5
2. They represent values between zero and the smallest normalised magnitude, making the transition toward zero gradual. §4.5
3. Positive or negative zero, according to the sign bit. §4.5

Explain it

1. The rightmost place is position zero, and any base raised to zero is 1. Its bit contributes either zero or one to the value.
2. Each octal digit maps to exactly three bits and each hexadecimal digit to four. Grouping bits preserves their value while shortening the written form.
3. Both encodings use a sign bit and a separate magnitude/complement pattern for zero. That creates distinct positive-zero and negative-zero bit patterns.
4. With n bits, one pattern is used for zero and the remaining patterns cannot be divided evenly into equal positive and negative ranges. Two's complement assigns the extra pattern to the negative side.
5. Fixed-width addition retains only n result bits, so carry beyond the word is discarded. Signed overflow depends on the signs of the inputs and result, not on that discarded bit.
6. Each four-bit BCD group can be read as one decimal digit, but four bits are used per decimal digit even when ordinary binary could represent the whole number with fewer bits.
7. The exponent scales a significand by powers of the base, expanding which magnitudes can be represented. Precision depends chiefly on how many significand bits distinguish values at a given scale.
8. At a larger exponent, the same significand step is multiplied by a larger power of 2. Therefore adjacent representable values are farther apart.
9. A finite binary fraction has a denominator that is a power of 2 after reduction. One tenth has a factor of 5 in its denominator, so its binary expansion repeats.
10. Only values with the same exponent have significands describing the same scale. Alignment expresses both at a shared scale before combining them.
11. Alignment shifts the smaller significand to the right. If the exponent difference is large, its significant bits may fall beyond the available precision.
12. Every normalised binary number begins with 1 before the point, so the format can infer that bit and use its storage for another fraction bit.
13. A subnormal represents a non-zero value smaller than the minimum normalised magnitude, close to zero.
14. Absolute or relative error that is acceptable depends on the scale and purpose of the calculation. One fixed tolerance may be too strict for large values or too loose for small ones.
15. Spacing scales with the exponent, so a fixed absolute difference is a different fraction of values at different magnitudes. Machine epsilon measures spacing near 1; the relative rounding model expresses error as a fraction of the input magnitude.
16. For positive floating-point x and y with $y/2 \leq x \leq 2y$, their difference is exactly representable under the format assumptions. Any rounding that already changed x or y can still affect the difference.

Mixed practice

1. $37=32+4+1$, so 100101_2. §3.1
2. $1/2+1/8+1/16=0.6875$. §4.2
3. Unsigned value $14-16=-2$. §3.2
4. 24 bits: the 23 stored fraction bits plus the implicit leading 1. §4.4
5. Four bits, since $16=2^4$. §3.1
6. $1.01_2\times2^{-4}$. The point moves four places right to make the significand 1.01, so the exponent is -4. §4.1
7. 7 is 0111 and 3 is 0011, so 73 is 0111 0011 in BCD. §3.2
8. $0.375=3/8=0.011_2$. §4.2
9. $2^7=128$ code values. §3.3
10. The exponent field. §4.1
11. $1011+0110=10001$. As an unsigned five-bit result it is 17. §3.4
12. Multiply significands: 6. Add exponents: -2. Result 6×10^{-2}, normalised as 6×10^{-2}. §4.3
13. -8 through $+7$. §3.2
14. $7.2\times10^5=0.072\times10^7$; shift the significand two places right as the exponent rises by two. §4.3
15. Group 101 101; each group is 5, so 55_8. §3.1
16. 52 stored fraction bits. §4.4
17. Pattern 110 is 6; $6-4=2$. §3.2
18. Negative infinity: all-ones exponent and zero fraction mean infinity; sign bit 1 makes it negative. §4.5
19. $29=5\times5+4$, so quotient 5 and remainder 4; $0\leq4<5$. §3.4
20. $1/4+1/8+1/32=0.40625$. §4.2
21. Invert 0011 to 1100 and add 1: 1101, which is -3. §3.2
22. Align $2.1\times10^2=0.21\times10^3$. Sum 8.67×10^3; to two significant digits this is 8.7×10^3. §4.3
23. 1110 1010 maps to E A, so EA_{16}. §3.1
24. 127. §4.4
25. The machine keeps only the fixed n result bits. Carry-out is outside that word and does not alone indicate signed overflow. §3.5
26. $0.625=1/2+1/8=0.101_2$. §4.2
27. BCD: 0101 1001. Ordinary binary: 111011. §3.2
28. $130-127=3$. §4.4
29. -6 is 1010; invert and add 1 to get $+6=0110$. Then $0111+0110=1101$, which decodes as -3. The exact result is $+13$, outside $-8\ldots+7$, so overflow occurs. §3.5
30. Truncation always discards in the same direction for positive values and can introduce a biased error. §4.3
31. Unicode code points 0–127 correspond to ASCII characters. §3.3

32. A subnormal number. The exponent is zero and the fraction is non-zero. §4.5
33. 101 is 5 and 11 is 3; $5\times3=15=1111_2$. §3.4
34. $1.25\times8=10$. §4.1
35. 0101+0011=1000. The exact result is +8, outside the four-bit range −8…+7; the two positive inputs produced a negative-sign pattern, so it overflows. §3.5
36. 64 bits: one sign, eleven exponent, and 52 fraction bits. §4.4
37. $7-8=-1$. §3.2
38. For example, treat a as close to b when $|a-b|<\varepsilon$, with ε chosen for the expected scale and precision. §4.3
39. BCD keeps each decimal digit directly encoded, which can simplify decimal display or input/output. §3.2
40. $1+1/2+1/8=1.625$. §4.2
41. $1.25=1.01_2$ is a finite binary fraction, and multiplying by 2^3 only shifts the binary point. §4.2
42. $\pm\infty$ and NaN, respectively; the first divides a non-zero value by zero, while $0\div0$ has no real result. §4.5
43. Add the exponents and multiply the significands, then normalise if needed. §4.3
44. Positive infinity. §4.5
45. For $\beta=2$ and $p=24$, $\varepsilon_{mach}=2^{-23}\approx1.19\times10^{-7}$. Under round-to-nearest, $u=\varepsilon_{mach}/2=2^{-24}\approx5.96\times10^{-8}$. §4.5
46. $53=32+16+4+1=110101_2$. §3.1
47. $0.5\le1.5\le2$, so the condition holds. The difference $1.5-1.0=0.5$ is exact. §4.5

Exam-style problems

1. +5 is 0101. Sign-and-magnitude is 1101. One's complement is 1010, the inversion of 0101. Two's complement is 1011, the inversion plus 1. §3.2
2. For four bits, sign-and-magnitude and one's complement both range from −7 to +7 and have two zero patterns. Two's complement ranges from −8 to +7 and has one zero. §3.2
3. Negation and subtraction use the same fixed-width adder: invert the subtrahend and add one. Its one extra negative value is usually a small trade-off for simpler arithmetic hardware. §3.2 §3.5

Exam-style problems

1. −5 is 1011 and +3 is 0011. Addition gives 1110 with no carry-out. §3.5
2. 1110 is 14 unsigned; subtract 16 to decode −2. The exact sum is also −2. §3.2 §3.5
3. No overflow: the exact result −2 lies in −8…+7. Carry-out is not the signed-overflow criterion; compare the signs or the exact result to the range. §3.5

Exam-style problems

1. $13.25{=}1101.01_2$, since $13{=}1101_2$ and $0.25{=}0.01_2$. §4.2
2. Normalise: $1.10101{\times}2^3$. The sign is 0. The stored fraction is 10101000000000000000000 after padding to 23 bits. §4.4
3. Biased exponent is $3{+}127{=}130{=}10000010_2$. The pattern is 0 | 10000010 | 10101000000000000000000. Decoding restores $1.10101_2{\times}2^3{=}13.25$. §4.4

Exam-style problems

1. $5.6{\times}10^2{=}0.056{\times}10^4$. §4.3
2. $(3.4{+}0.056){\times}10^4{=}3.456{\times}10^4$, which rounds to $3.5{\times}10^4$. §4.3
3. The exact sum is 34560 and the rounded result is 35000. The smaller addend contributes digits beyond the two-digit precision, so rounding changes the stored result. §4.3

Personal study guide based on the supplied APDR Chapter 3 and Chapter 4 lecture notes and "APDR lecture 2 extended", slides 1–23. Written in my own words; no lecture pages or figures are reproduced. Not an official FAU publication; check course details against the current notes.